Toward Practical Superconducting Accelerators for Machine Learning Using U-SFQ

PATRICIA GONZALEZ-GUERRERO, KYLIE HUCH, NIRMALENDU PATRA, THOM POPOVICI, and GEORGE MICHELOGIANNAKIS, Lawrence Berkeley National Laboratory, Berkeley, California, USA

Most popular superconducting circuits operate on information carried by ps-wide, μV-tall, single flux quantum (SFQ) pulses. These circuits can operate at frequencies of hundreds of GHz with orders of magnitude lower switching energy than complementary-metal-oxide-semiconductors (CMOS). However, under the stringent area constraints of modern superconductor technologies, fully-fledged, CMOS-inspired superconducting architectures cannot be fabricated at large scales. Unary SFQ (U-SFQ) is an alternative computing paradigm that can address these area constraints. In U-SFQ, information is mapped to a combination of streams of SFQ pulses and in the temporal domain. In this work, we extend U-SFQ to introduce novel building blocks such as a multiplier and an accumulator. These blocks reduce area and power consumption by 2× and 4× compared with previously proposed U-SFQ building blocks and yield at least 97% area savings compared with binary approaches. Using these multiplier and adder, we propose a U-SFQ Convolutional Neural Network (CNN) hardware accelerator capable of comparable peak performance with state-of-the-art superconducting binary approach (B-SFQ) in 32× less area. CNNs can operate with 5–8 bits of resolution with no significant degradation in classification accuracy. For 5 bits of resolution, our proposed accelerator yields 5× to 63× better performance than CMOS and 15× to 173× better area efficiency than B-SFQ.

CCS Concepts: • **Hardware → Emerging technologies**; **Analysis and design of emerging devices and systems**; **Emerging architectures**;

Additional Key Words and Phrases: Superconducting digital computing, convolutional neural networks, unary computing, SFQ

ACM Reference format:
Patricia Gonzalez-Guerrero, Kylie Huch, Nirmalendu Patra, Thom Popovici, and George Michelogiannakis. 2024. Toward Practical Superconducting Accelerators for Machine Learning Using U-SFQ. *ACM J. Emerg. Technol. Comput. Syst.* 20, 2, Article 7 (June 2024), 22 pages.
https://doi.org/10.1145/3653073

This work was supported by the Director, Office of Science, of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231.

Authors' Contact Information: Patricia Gonzalez-Guerrero (Corresponding author), Michelogiannakis, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, California, USA; e-mail: lg4er@lbl.gov; Kylie Huch, Michelogiannakis, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, California, USA; e-mail: kyliehuch@lbl.gov; Nirmalendu Patra, Michelogiannakis, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, California, USA; e-mail: nbp@lbl.gov; Thom Popovici, Michelogiannakis, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, California, USA; e-mail: dtpopovici@lbl.gov; George Michelogiannakis, Lawrence Berkeley National Laboratory, 1 Cyclotron Road, Berkeley, California, USA; e-mail: mihelog@lbl.gov.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1550-4840/2024/6-ART7
https://doi.org/10.1145/3653073

1 INTRODUCTION

Superconducting digital computing technology has enabled **single flux quantum (SFQ)** logic gates that operate at frequencies of up to 770 GHz [7, 22, 23, 27, 32] while keeping the switching energy six orders of magnitude lower than current **complementary-metal-oxide-semiconductors (CMOS)** processes [33, 49].

This impressive clocking speed and energy efficiency makes superconductors attractive for algorithms that require high-performance computing, such as **Convolutional Neural Network (CNN)** hardware acceleration [6, 20]. However, building realistic-scale CNN accelerators in superconducting technology is a monumental challenge. Manufacturing limitations restrict the number of active devices, also known as Josephson Junctions (JJs) (Figure 1(a)), in superconducting chips to just a few tens of thousands per die, yielding an extremely low area density [26]. The largest superconducting prototype reported in literature (~22K JJs [2, 35]) can only integrate a single **binary SFQ (B-SFQ)** multiplier (17K JJs [35]). In contrast, a state-of-the-art CMOS-based CNN hardware accelerator [21] integrates 65,536 parallel multipliers and adders on a single chip.

Although we expect superconducting technology to mature such that the device density significantly improves [5], there are no clear indications that we will be able to integrate 65,536 multipliers in a superconducting chip [45, 49].

Non-conventional computing paradigms might be key to leverage the unique properties of superconducting technology while addressing the area density constrains [6, 14, 50]. In particular, using **Unary SFQ (U-SFQ)** [14], basic computing elements require minimal area. For example, a U-SFQ multiplier requires only 10 JJs. That is 99% fewer JJs than a traditional B-SFQ multiplier. U-SFQ area efficiency comes from its peculiar data representation that maps data to either pulse streams or temporal data also known as **race logic (RL)**, and how those two representations interact. Besides area efficiency, U-SFQ leverages the good properties of other non-conventional, approximate-computing-paradigms such as stochastic computing or computing with streams [13, 15, 16, 28, 53]. U-SFQ is extremely robust to noise and errors due to the uniform and small weight $1/2^b$ associated with each SFQ pulse, where b is the bit resolution. A missing pulse has a negligible effect in the computation's accuracy, specially when b is large. Note that in contrast with stochastic computing, U-SFQ is not an approximate computing paradigm. Thus, compared with a fixed-point-binary approach there is no degradation in the algorithm's accuracy.

CNN hardware accelerators can benefit from U-SFQ's minimum area footprint due to the extreme parallelism CNNs motivate. However, using U-SFQ in a typical, CMOS-inspired CNN architecture is not practical. U-SFQ brings up new architectural challenges and design choices due to its unique property of trading area for latency. This work is the first one to explore a CNN hardware accelerator tailored to U-SFQ. Our contributions are summarized as follows:

- We extend U-SFQ to support magnitude-sign representation for RL. This extension enables us to introduce new U-SFQ blocks such as a multiplier, an accumulator, and a **processing element (PE)**. These new blocks consume 2–4× less area and power than previously proposed U-SFQ blocks [14]. Compared with their superconducting binary counterparts, the proposed blocks yield at least 96% savings in area and power consumption (Section 3.1).
- We study the implications of using U-SFQ to build a PE array for CNN hardware acceleration. In particular, we discuss the effect of variable bit resolution, area efficiency, power efficiency, classification accuracy, and memory bandwidth (Section 3.2).
- We propose an area-efficient U-SFQ **three-dimensional (3D)** PE array that can yield a peak performance comparable with that of a state-of-the-art B-SFQ in 32× less area (Section 3.3).
- We evaluate the proposed U-SFQ 3D PE array and compare it against state-of-the-art B-SFQ and CMOS CNN hardware accelerators (Section 4).

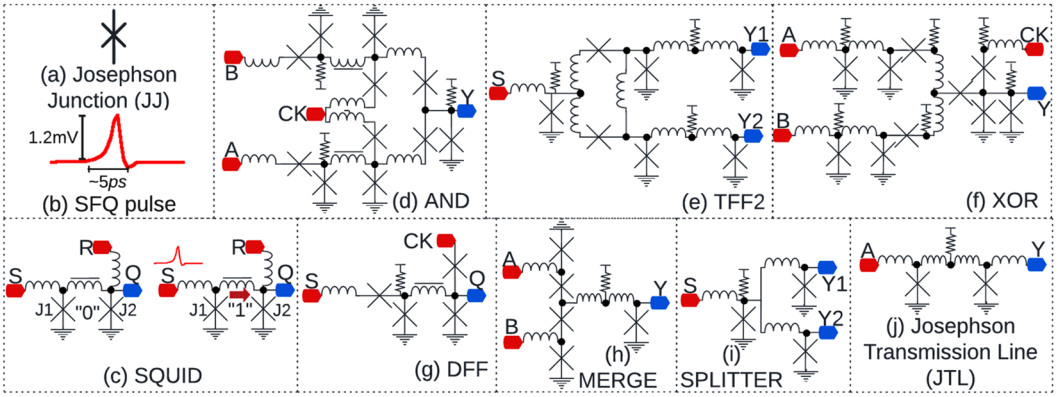


Fig. 1. (a) The **Josephson Junction (JJ)** symbol used in circuit diagrams. (b) A SFQ pulse. The pulse amplitude and width are 1.2 mV and 5 ps, respectively for the *MIT-LL SFQ5ee* 10 kA/cm^2 fabrication process. (c) **Superconducting QUantum Interference Device (SQUID)**. (d–j) Superconducting **Rapid Single Flux Quantum (RSFQ)** building cells relevant to this study [27, 34, 55]. Input ports in red, output ports in blue. (d) AND gate. (e) **Toggle Flip-Flop (TFF)** with two outputs. (f) **Exclusive OR (XOR)** gate. (g) **Data Flip-Flop (DFF)**. (h) Merger (M). (i) Splitter (S). (j) **Josephson Transmission Line (JTL)**.

Our efficient 3D U-SFQ CNN PE array requires 32× fewer JJs than a B-SFQ approach. Evaluating this array with CNN benchmarks such as Alexnet, ResNet, VGG16, and GoogleNet, we find that U-SFQ yields speedups over CMOS and better area efficiency over B-SFQ for computing with less than 7 bits of resolution. Note that low-bit quantization of weights and **input feature maps (ifmap)** has been demonstrated for 5 [54] and variable bit widths [3]. In these cases, we obtain a 5× to 63× speedup over CMOS and 15× to 173× better area efficiency over B-SFQ depending on the benchmark. Using an **energy-efficient superconducting logic family (ERSFQ)**, U-SFQ is up to 26× more power efficient when compared with CMOS, even when the cooling cost is included.

2 MOTIVATION AND RELATED WORK

Superconducting circuits started as an early competitor for today's well-established semiconductor industry. But as the semiconductor industry thrived, the enthusiasm for superconducting computing dissipated due to the need for expensive refrigeration. However, with the slowdown of Moore's law, superconductors have gained momentum as a candidate for preserving computational scaling. In this section, we explain the basics of superconducting digital computing and go through its challenges. We review U-SFQ, an area-efficient superconducting non-conventional computing paradigm. We also briefly review state-of-the-art CNN hardware accelerators that will serve as a baseline for our evaluation.

2.1 Superconducting Computing

Superconductivity is the property of some metals, such as aluminum or also known as superconductors, can be used to build the JJ [40], a switch or active device that is the heart of superconducting computing.

The JJ (Figure 1(a)) allows current through with no resistance up to a critical current I_c. Reaching I_c causes the JJ to temporarily switch to a resistive state, emitting a ps-wide, μV-tall voltage pulse [30] shown in Figure 1(b). Typically, the JJ is placed in a superconducting loop (Figure 1(c)), also known as Superconducting QUantum Interference Device (SQUID), where the magnetic flux ϕ can take only integer multiple values of a SFQ $\phi_0 = \frac{h}{2e} = 2.07 \times 10^{-15}$ Wb. The amount of flux in the

circuit can be controlled by switching the JJ between the resistive and superconducting states. A SQUID has two stable stationary states. An input pulse at the S input creates a clockwise current flow. An input pulse at the R input causes J2 to transition. Therefore, the loop reverts to its initial state and an output pulse is generated at the SQUID output (Q). Analogous to CMOS where a digital "1" is the voltage level Vdd and the digital "0" is the voltage level 0, in superconducting computing the presence of SFQ in the loop represents a digital "1," while its absence a digital "0."

2.1.1 RSFQ Logic Family. RSFQ is a logic family that has evolved to offer a rich set of synchronous logic gates such as AND or Data Flip-Flop (DFF) (Figure 1(d) and (g)); and interconnection cells such as the splitter and Josephson Transmission Line (JTL) (Figure 1(h)–(j)). In general, synchronous logic gates are built as modified versions of the SQUID. For example, the DFF is the SQUID, while the AND gate can be seen as two DFFs sharing a **clock (CK)** signal in parallel. With these gates, several state-of-the-art superconducting computing designs such as adders, multipliers [1, 11, 35], processors [12, 22], and hardware accelerators [20, 49] have been demonstrated.

2.1.2 Superconducting Computing Challenges. Superconducting computing faces five significant challenges: (1) limited area density, (2) expensive CK distribution networks, (3) high static power consumption, (4) cooling costs, and (5) low memory density.

(1) Limited Area Density: JJ feature sizes range from 250 nm to 800 nm [26], which is 35× to 100× larger than current 7 nm to 5 nm CMOS technologies. Assuming there are significant markets to fund superconducting electronic development [17], we expect that JJ density will improve with evolution in materials, specialized circuit design techniques, cryogenic test methodologies, and **Electronic Design Automation (EDA)** algorithms [26]. However, there is currently a density gap between SFQ technology and CMOS [49] for two reasons: (1) JJ logic gates have limited output driving capacity. For example, standard CMOS gates have a fan-out of four, whereas JJ-based logic gates usually can drive at most one output without requiring extra output drivers and (2) these output drivers, also known as JTLs and Passive Transmission Lines cost at least two JJs per connection gate.

(2) Clock Distribution: Unlike CMOS, the majority of RSFQ gates for binary computing are synchronous. This requires a robust CK distribution network that reaches almost every cell in the design, must be made bullet-proofed against variations, and must be capable of maintaining the correct operation in the multi-gigahertz frequencies. This expensive network exacerbates the area density issue and increases the design complexity of superconducting RSFQ circuits.

(3) RSFQ Static Power Consumption: Although RSFQ is one of the most developed superconducting families, it suffers from high static power consumption due to the resistive current distribution network used for biasing the JJs (see Figure 1). To eliminate static power, the ERSFQ logic family was proposed. ERSFQ gates are similar to RSFQ gates, except that in ERSFQ bias resistors are replaced by limiting JJs and series inductances [33]. ERSFQ causes a slight (1.4×) increment in area due to the extra JJs used for power distribution and bias inductors. Another promising logic family is eSFQ [52]. In this case, the large bias inductors are eliminated by forcing synchronous phase compensation at gate bias terminals. Additionally, in eSFQ, the feeding JTLs are combined with the CK JTLs. The major drawback of this technology is the increased design cost compared with RSFQ. Table 1 compares the performance of these energy-efficient logic families with RSFQ. In our work, we design and evaluate our architecture using the RSFQ logic family.

(4) Cooling: Superconducting electronics require refrigeration below 4 K. The refrigerator used for cooling down the circuits consumes extra energy which contributes to the overall power consumption of a superconducting circuits solution. The cooling cost can be roughly estimated by multiplying the circuit power consumption by 400 [20, 56] assuming the power efficiency of 400 W/W for a helium liquefier refrigeration system [17]. However, note that there are systems that

Table 1. Comparison of ERSFQ and eSFQ against the RSFQ [33]

	ERSFQ	**eSFQ**
P_S[a]	Zero	Zero
P_D[a]	2×	1.1×
JJ count	1.4×	1.4×
Cell design	Same	Modified
Max CK	Slightly lower	Same
Switching speed	Same	Same

[a] P_S, P_D are static and dynamic power, respectively.

already operate in refrigerated environments, such as quantum computers [29] and outer space electronics [26]. These applications can immediately benefit from superconducting high-speed and low-energy consumption. In this case, refrigeration does not incur in extra costs, because it is already intrinsic to the application, and we assume that refrigeration comes for free or at a negligible cost. Thus, we report power with and without cooling costs.

(5) Low Memory Density: On-chip and off-chip memory for superconducting technology have been a long-standing issue for applications that require large amounts of data accessed fast and energy efficiently, including CNNs [20, 48, 51]. The authors in [20] propose to use SFQ shift register-based memories for speed and practical reasons. However, when comparing the shift register memory approach with an ideal memory system that does not cause stalls, the inference latency is degraded by 17× due to the recurrent shifts required to search for data in memory [56]. Moreover, when the authors in [56] compare the effect on the performance of using other types of cryogenic memories such as (1) Vortex-Transition-Memory, (2) **Josephson-CMOS SRAM (SRAM)**, (3) **Magnetic RAM (MRAM)**, and (4) **Superconducting Nanowire RAM (SNM)**, the results are not encouraging. Since SRAM, MRAM, and SNM have long read and write latencies (0.1–4 ns), they increase the inference latency by 5× compared to a shift register memory system. Exacerbated by the latency increment, the energy consumption of all the cryogenic memory technologies is larger than the shift register-based memory. Also, memory peripherals in SFQ, specially decoders, limit the area efficiency of all the memory technologies. To address the memory challenges, the authors in [56] propose a heterogeneous memory architecture that combines shift arrays for sequential accesses with a pipelined RAM array.

Although our work focuses on the potential of U-SFQ for accelerating the computation part of a CNN acceleration, we acknowledge that evaluating the cost and performance of on-chip memory is equally important. Previous work briefly introduced superconducting memory elements for U-SFQ [4, 14]. These memory elements, based on shift registers, **Non-Destructive Read Outputs (NDROs)**, and analog-buffers, are good candidates for efficient U-SFQ on-chip memory. For our evaluation, we assume the same on-chip memory capacity than state-of-the-art B-SFQ [20] (~49 MB). However, this is mainly unexplored land that deserves a separate discussion on its own and it is out of the scope of this article.

2.2 U-SFQ Logic

Inspired by the natural occurrence of pulses in SFQ logic, recent work explored U-SFQ, an alternative data representation that combines SFQ pulse streams with SFQ temporal data representation, also known as RL. U-SFQ alleviates the stringent superconducting area constraints by trading latency for area while still enabling efficient and small footprint building blocks [14].

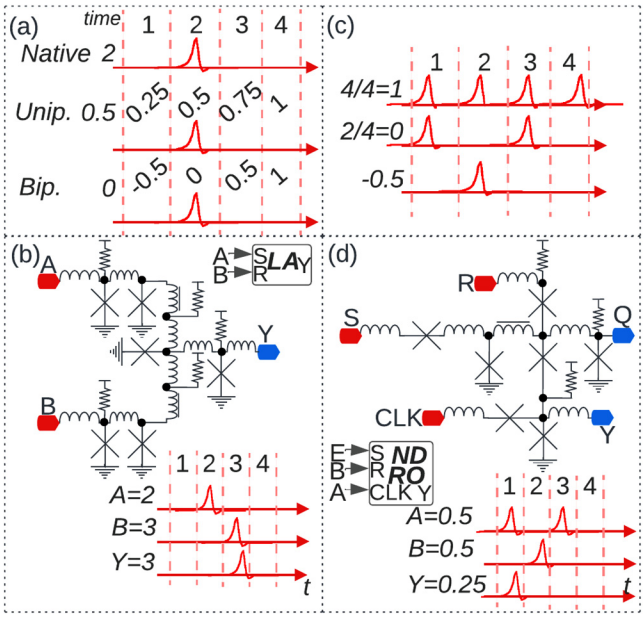


Fig. 2. (a) RSFQ coincidence gate, used to implement the **last-arrival (LA)** RL logic gate [50]. Example of computing the maximum using an LA for RL numbers $A = 2$, $B = 3$. The output $Y = 3$ is the maximum between 2 and 3 ($max(2, 3)$). (b) Example of U-SFQ multiplication. A = 0.5 is a pulse stream with half the maximum frequency, and $B = 0.5$ is represented in RL. $Y = 0.25$ is a pulse stream containing all the pulses in A before B arrives. This operation can be implemented with an NDRO cell [27]. E (not shown) marks the beginning of the computing time. (c) Examples of the same RL signal for three different encodings: native RL, unipolar RL, and bipolar RL. (d) Examples of bipolar pulse streams. For the last stream $0.5 = \frac{-3+1}{4}$.

2.2.1 RL. In RL, information is encoded as a delay from a reference signal. The computation outcome depends on the relative propagation time of signals injected into a circuit [28]. For example, Figure 2(b) demonstrates computing the maximum between two RL signals using an SFQ coincidence gate which requires only 6 JJs. The coincidence gate generates a pulse at output port Y when both A and B arrive in any order, thus behaving as an LA RL gate [50].

Because RL encodes data in the temporal domain, it is well-suited for dynamic programming [28]. However, basic arithmetic operations such as addition and multiplication are expensive. To encode numerical values in RL, the computation time, also known as epoch, can be divided into time slots. Figure 2(a) shows an RL epoch that consists of four time slots. The SFQ pulse arrives at the time slot identified as $Id = 2$ to represent number two. Each time slot (Id) can also be divided by the epoch's maximum number of time slots. This results in a *unipolar* numerical representation between 0 and 1. To represent negative numbers, an RL *bipolar* representation can be obtained by scaling and shifting the unipolar Id such that $Id_b = 2 \times Id_u - 1$, where Id_u is the unipolar Id and Id_b is the equivalent bipolar Id [14]. Figure 2(a) shows examples of unipolar and bipolar RL representations for an epoch with 4 time slots, that is, 2 bits of resolution.

2.2.2 SFQ Pulse-Streams Data Representation. In the pulse-stream data representation, a number p is mapped to the frequency of a periodic train of SFQ pulses. For a *unipolar* pulse-stream data representation, the maximum rate, or equivalently the maximum number of pulses N_{max} in an epoch, is mapped to 1 and the absence of all pulses in an epoch to 0. Thus, $p = \frac{n}{N_{max}}$, where n is the number of pulses during a computing epoch. Note that each pulse has an associated weight of

$\frac{1}{N_{max}}$. For a *bipolar* data representation, $p_b = 2 \times p_u - 1$, where p_u is the unipolar representation and p_b is the equivalent in a bipolar representation [14]. Figure 2(c) shows examples of bipolar pulse streams.

Building blocks such as multipliers and adders for U-SFQ have been proposed [14]. For example, a multiplier can be implemented using an NDRO cell [27] (Figure 2(d)). Signal E sets (enables) the NDRO indicating the beginning of the epoch. Before a pulse arrives at *B*, pulses at *A* propagate to output *Y*. The arrival of *B* resets the NDRO, suppressing the propagation of pulses from *A* for the remaining of the epoch. This multiplier enables more than 90% savings in the area and yields a latency comparable with an SFQ binary multiplier for less than 8 bits of resolution [14]. See Section 3.1.1 for more details.

2.3 CNN Hardware Acceleration

CNNs are attractive for their high performance in image segmentation, classification, detection, and retrieval-related tasks. Moreover, CNNs are circuit-oriented architectures, which readily translate to low-power and efficient hardware accelerators [31]. A CNN is composed of multiple stages or layers that can be of three different types: (a) convolutional layers, (b) non-linear processing layers, and (c) subsampling layers [24]. A convolutional layer is characterized by parameters such as ifmap height and width ($H \times W$), the number of ifmaps (Z), kernel height and width ($K \times K$), **output feature map (ofmap)** height and width ($R \times C$), and the number of channels (M). Since convolutions account for over 90% of the runtime and energy consumption, most of the hardware acceleration efforts focus on this layer [47].

Many typical CNN hardware accelerators are based on **two-dimensional (2D)** systolic arrays [43]. A 2D systolic array consists of regularly and locally interconnected PEs operating in parallel, whose dynamic behavior is determined by the cell connectivity pattern (neighborhood extent) and a set of configurable parameters [31]. Each PE has a multiplier, an accumulator, and local memory at a minimum.

One of the most popular CNN hardware accelerators is the **tensor processing unit (TPU)** implemented in CMOS technology [21]. The TPU features an array of 256×256 PEs and a peak performance of 45 **tera-multiply-accumulate-per-second (TMAC/s)**. Given the high levels of integration and performance, the TPU has become a baseline for comparison for research on CNN hardware acceleration; thus, in this work, we use the TPU PE array as a baseline for comparison with CMOS.

Recent work explored a **superconductor Neural Processing Unit (superNPU)** for CNN acceleration [20]. The heart of this accelerator is an array of 64×256 PEs; superNPU yields a theoretical peak performance of 842 TMAC/s. Given the large number of JJs required for such a system, the superNPU has never been impractical. However, we use superNPU as the B-SFQ baseline for comparison.

3 U-SFQ CNN HARDWARE ACCELERATOR

To leverage the unique properties of U-SFQ, we perform an architecture exploration to propose a 3D PE array for CNN hardware acceleration optimized for U-SFQ. Three significant factors dictate the characteristics of any PE array: (a) the PE, which performs the mathematical computation; (b) the dataflow, which indicates how data traverse the array to complete a convolution; and (c) on-chip and off-chip memory capacity. For a U-SFQ PE array, we also have (d) bit resolution. Although (a), (b), and (c) have been widely studied for CMOS-based accelerators [10, 46, 47], and some studies have shown promise in B-SFQ [20], we need to understand the effect of each parameter when using a U-SFQ computing paradigm. Therefore, this section introduces a novel U-SFQ PE for the 3D array

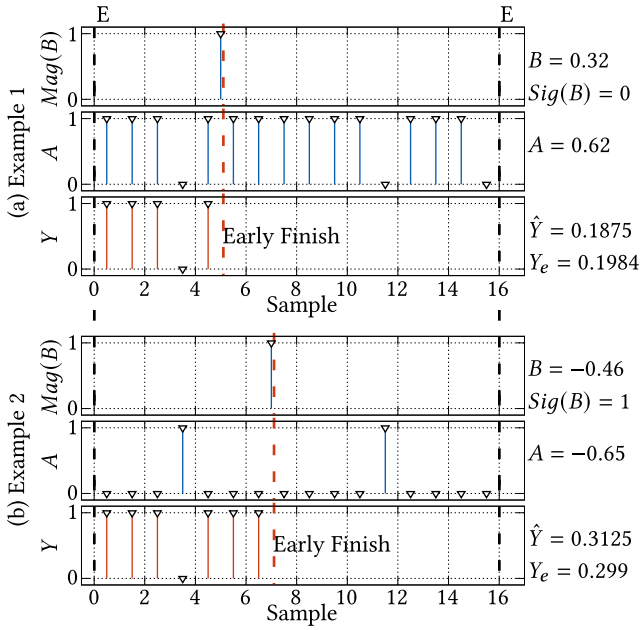


Fig. 3. Multiplication examples using bipolar multiplier II for 4 bits of resolution. E signals the beginning of each new epoch. B is represented as a magnitude-sign RL value. $Sig(B) = 1$, not shown, is a pulse stream with maximum frequency and $Sig(B) = 0$, not shown, is the absence of pulses. A is a pulse stream. Y is the U-SFQ multiplier output. Y_e is the expected multiplication result from stream Y. The arrival of a pulse in B marks the end of the computation. Depending on the value of B, this may allow an early finish of the epoch. In Example 1, A = 0.62 and B = 0.32. In Example 2, A = −0.65 and B = −0.46.

and discusses its performance in isolation. Then, we do an architecture exploration of (b), (c), and (d) to find the optimum 3D PE array that maximizes area efficiency and performance.

3.1 The U-SFQ PE

A PE typically comprises a multiplier, an adder, and a relatively small register file. Previous work proposed an area-efficient U-SFQ multiplier and adder [14]. However, the multiplier is affected by long latency and the adder scales the output (divided by two), which affects the overall accuracy of the computation. To address these, we propose a novel multiplier and multiply accumulation unit that reduces the computation latency by half and avoids format conversion and scaling to keep the accuracy of the computation.

3.1.1 Multiplication. To multiply numbers A, B we can represent A as a bipolar SFQ pulse stream [14]. Inspired by [44] we can use an RL magnitude-sign representation for B. B's magnitude ($Mag(B)$) is a RL pulse. B's sign ($Sign(B)$) is an SFQ pulse stream encoding a "1" for $B < 0$ and a "0" for $B > 0$. To achieve multiplication $Sign(B)$ is XORed with input A and the computation finishes as soon as $Mag(B)$ arrives. Figure 3 shows multiplication examples using the proposed multiplication scheme. In these examples, we show 4 bits of resolution for each operand, i.e., an epoch E can have a maximum of 16 pulses. In the first example $A = 0.62$ and $B = 0.32$. $Sign(B) = 0$ (not shown) is simply the absence of pulses. Note that the computation finishes as soon as $Mag(B)$ arrives at time slot i_f. The multiplication result is obtained as $\hat{Y} = \frac{\sum_i^{i_f} Y_i}{N_{max}} = \frac{4-1}{16} = 0.1875$ with $Y_i = \pm 1$ depending on the presence (+1) or absence of the SFQ pulse (−1), and N_{max} is the maximum

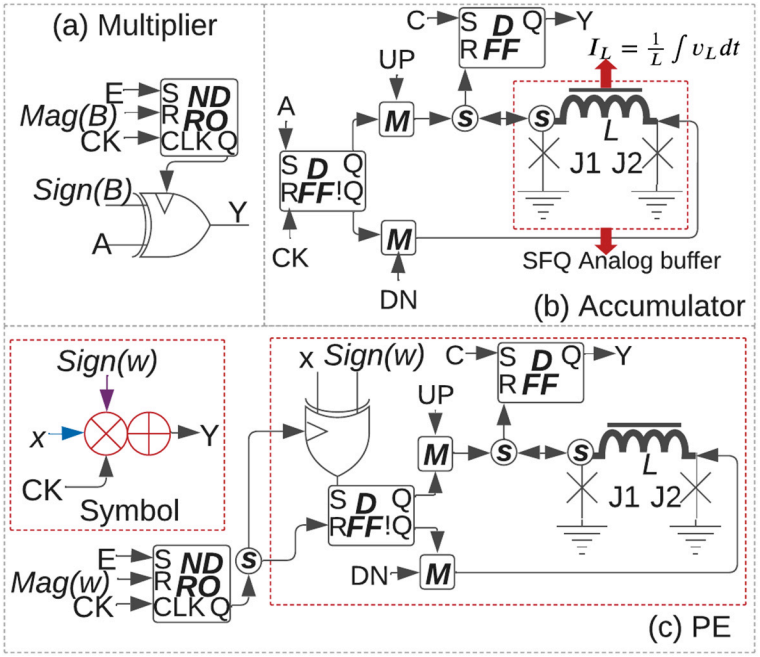


Fig. 4. Circuit implementations for (a) multiplier, (b) accumulator, and (c) PE and PE symbol. M is a merger and S a splitter.

number of pulses. In the second example, we have six pulses $\hat{Y} = \frac{6-1}{16} = 0.3125$. Note we use a fixed point data representation, thus similar to a binary approach there are truncation errors.

The computation latency is dictated by $Mag(B)$ and thus is data dependent. The worst case latency is $t_{max} = 2^{b-1} \times t_s$, with b the bit resolution and t_s the time slot. This worst case occurs when $B = 1$ because the multiplication requires all the time slots that is the computation epoch.

Figure 4(a) shows the circuit implementation of the proposed multiplier. We use an RSFQ Exclusive OR (XOR) gate combined with an NDRO. The NDRO disables the CK at the arrival of $Mag(B)$, enabling early termination. We implement the proposed multiplier using the open-source *MIT-LL SFQ5ee* 10 kA/cm^2 process and simulate it with the open-source software WRSPICE. Table 2 compares the area in number of JJs and static power consumption of the proposed multiplier with a previously proposed U-SFQ multiplier [14] and a state-of-the-art bit-parallel B-SFQ multiplier [35]. Compared with the previously reported U-SFQ multiplier, this work reduces the area and power consumption by 2× and 4×, respectively. Compared with B-SFQ, our multiplier yields 99% savings in area and power. Figure 5 shows the throughput in **giga-operations-per-second**[1] **(GOP/s)**, area efficiency in mega-operations-per-second-per-JJ (MOP/s/JJ), and power efficiency in **tera-operations-per-second-per-watt (TOP/s/W)**. For the U-SFQ approach, these metrics decrease with the bit-width increments because of the computing epoch lengthening. Our proposed multiplier yields better area and power efficiency at the expense of less throughput. The key for good throughput is many multipliers working in parallel, as we will discuss in the following sections.

3.1.2 Accumulation. For accumulation, we propose to use an integrator-based analog buffer shown in Figure 4(b). This buffer uses an inductor to integrate incoming pulses from stream A

[1]Operations are fixed-point multiplications and additions for multiplier and adder, respectively.

Table 2. Area and Power for Adder, Multiplier, and PE. Binary Multiplier (BM). Unary Multiplier (UM). Binary Adder (BA). Unary Adder (UA).

Component	Area #JJs	P_S (mW)
Multiplier		
B-SFQ (BM [35])	17K	5.6
U-SFQ (UM [14])	63	0.04
U-SFQ (UM) (this work)	27	0.01
Adder		
B-SFQ (BA)	3.5K	1.39
U-SFQ (UA) (this work)	78	0.019
PE (for CNNs)		
U-SFQ	94	0.03
B-SFQ	20.5K	7

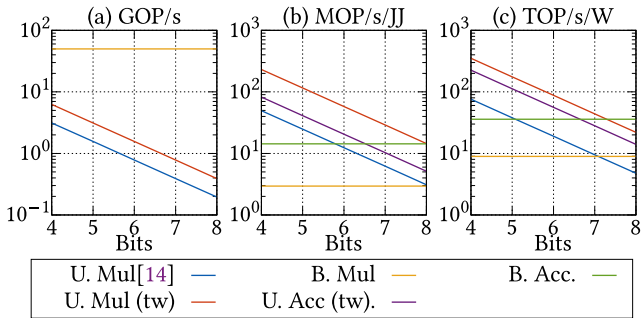


Fig. 5. Efficiency of U-SFQ (labeled as U.) and B-SFQ (labeled as B.) multipliers and accumulators as a function of bit resolution. (a) Throughput (GOP/s). (b) Area efficiency calculated as throughput-per-JJ (MOP/s/JJ). (c) Power efficiency calculated as throughput-per-Watt (TOP/s/W). tw, this work.

given the inductor's current–voltage relation $I_L = 1/L \int v_L dt$ and returns the accumulated result Y in RL format. Note that our multiplier's output is encoded as a pulse stream, which matches the accumulator input's format.

Figure 6 shows a WRSPICE simulation of an accumulator that receives inputs with 3 bits of resolution (i.e., epochs have eight time slots). Obtaining the accumulation result requires three steps: (1) biasing, (2) accumulation, and (3) reading.

(1) Biasing: The UP signal is a pulse stream used to bias the inductance current to a level that represents a zero in a bipolar representation. A zero is a pulse stream with half the maximum frequency for a bipolar representation. This enables the inductance to be decremented within the underflow limits.

(2) Accumulation: Once the accumulator is adequately biased, the accumulation period starts and can continue for several epochs. The number of epochs depends on the application. For example, in Figure 6, we accumulate for three epochs, with signal E marking the beginning of each epoch. Since the integrator must allow increments and decrements of its stored value to support the bipolar data representation, we use a **DFF with complementary outputs (DFFC)**. The DFFC generates a pulse at output Q when there is a pulse in input A, and a pulse at !Q in the absence of a pulse in input A. This way, we can connect Q and !Q at each extreme of the inductor to achieve the

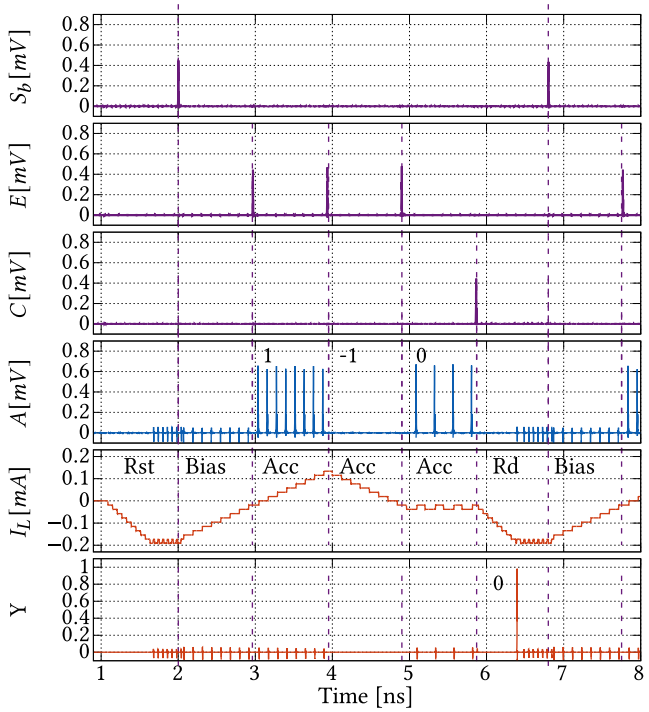


Fig. 6. WRSPICE simulation of the accumulator with an input/output resolution of 3 bits but an internal resolution of 4 bits to minimize the overflow effect. Signal S_b starts the biasing period where the current in the inductance is set to the equivalent of the 0 value. Each pulse in signal E starts a new accumulation epoch. In this example, the input A represents a $1, -1, 0$ at each of the three accumulation epochs. Signal C starts the reading epoch. The accumulation result ($Y = 1 - 1 + 0 = 0$) is obtained over three computation epochs and it is encoded in bipolar RL.

increment and decrement effect. When there is a pulse at Q ($!Q$), we increment (decrement) the current accumulated in the inductance by one flux-quantum.

(3) Reading: Once the accumulation finishes, we can obtain the accumulated value. The DN signal is a pulse stream used to read the information of the buffer by driving the inductance current to the minimum level. When the inductance current reaches the minimum current level, $J1$, which acts as a comparator, generates an RL pulse that represents the accumulated value and is equivalent to the time it takes for the current to be driven to the minimum level. Control pulse C sets a DFF state to 1. This DFF acts as a filter allowing the propagation of only the first pulse generated by $J1$ to output Y.

The control signals such as UP, DN, and C can be driven by a control unit. This control unit could be implemented using a combination of TFF-based counters, yet it depends on the overall system the accumulator belongs to.

We implement the proposed accumulator using the open-source *MIT-LL SFQ5ee* 10 kA/cm^2 **Process Design Kit (PDK)** and simulate it with the open-source software WRSPICE. Table 2 compares the area and power consumption of the proposed accumulator with a B-SFQ adder. The metrics for the B-SFQ adder are derived from an in-house bit-parallel model that derives power and area from JJ. We achieve at least 97% of savings in area and power consumption. Figure 5 shows the throughput, area efficiency, and power efficiency for the U-SFQ and B-SFQ approaches. Similar to the multiplier, these metrics decrease with an increment in bit resolution and U-SFQ trades area for latency. The proposed accumulator yields better area efficiency for less than 7 bits.

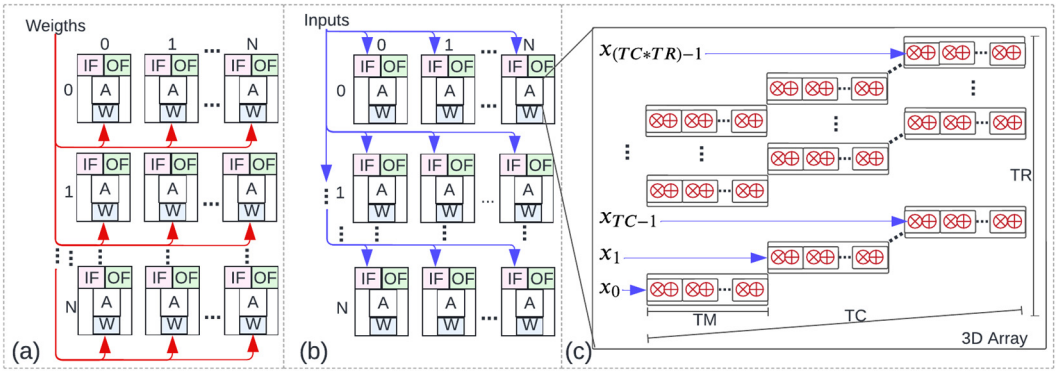


Fig. 7. Our proposed U-SFQ CNN hardware accelerator. (a and b) The accelerator consists of $N \times N$ subarrays to avoid impractical delays from signal distribution over a large array. Each subarray has a dedicated ifmap (IF), ofmap (OF), and weight (W) buffer. With $N = 3$ we have 16 subarrays. We can broadcast the weights (a) or the ifmap data (b) to operate in a $64 \times 64 \times 32$ or $16 \times 16 \times 512$ configuration, respectively. (c) The 3D PE array (A) comprises TR × TC × TM PEs.

3.1.3 The PE. Using the multiplier and accumulator introduced above, we propose a PE for CNN hardware acceleration shown in Figure 4(c). This PE calculates $Y = \sum w_i x_i$ where $Sign(w_i)$ is a pulse stream and $Mag(w_i)$ is a unipolar RL value. The input x_i is a pulse stream with p_x mapped to a stream rate. The PE's output Y is the multiply-accumulate result in RL format. We implement the U-SFQ PE using the open-source *MIT-LL SFQ5ee* 10 kA/cm^2 process and simulate it with the open-source software WRSPICE. Table 2 compares the area and power consumption for the U-SFQ and B-SFQ approaches. The U-SFQ PE yields at least 200× better area and power consumption than the binary approach. Since the PE usually works as part of an array, we will study performance versus bit width in the following sections.

3.2 Architecture Exploration to Obtain an Optimum U-SFQ PE Array

Using the PE introduced above, we propose a 3D array for CNN hardware acceleration. A 3D architecture instead of the more conventional 2D array is more appropriate for U-SFQ, as we will discuss in this section.

In a 2D array PEs are interconnected with their closest neighbors [42, 46, 47]. This way, data move between PEs in the vertical or horizontal direction each CK cycle. To keep this synchronous dataflow, PEs have local registers that store the data for one CK cycle before transmitting it to the neighboring PEs. These registers are usually overlooked in CMOS designs because they are relatively inexpensive and easily inferred by automated synthesis tools. Following a similar approach in U-SFQ is expensive because U-SFQ local memory takes up more area relative to its binary counterpart [4, 14].

A U-SFQ CNN hardware accelerator benefits from an architecture that avoids local registers associated with inter-PE communication, thus avoiding the local area overhead. This can be achieved by using a 3D PE array [38, 44]. In a 3D PE array, we map the ofmap dimensions row × columns × channels (R × C × M) to TR × TC × TM PEs. Figure 7(c) shows the array. To obtain a high reuse factor, TR × TC PEs use the same weight and TM PEs share the same input in a given computation time. In this way we spread data out to more PEs and need to fetch less data from memory. The question now, is what is the optimum 3D array, considering the U-SFQ effect in performance, classification accuracy, and memory bandwidth.

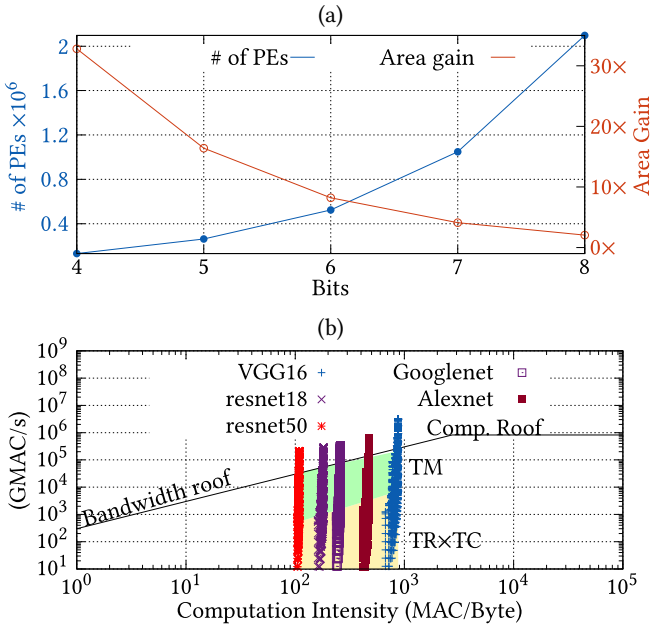


Fig. 8. (a) Number of U-SFQ PEs (# of PEs) required to obtain comparable B-SFQ peak performance versus bit resolution (left axis). U-SFQ area gains (in times ×) compared with B-SFQ for the same peak performance versus bit resolution (right axis). (b) Roofline analysis for a 3D U-SFQ PE array varying its dimensions TR ×TC ×TM.

3.2.1 Optimize by Peak Performance. A state-of-the-art B-SFQ CNN accelerator has the potential for an impressive peak performance of 812 TMAC/s [20]. To match this while keeping the area efficiency offered by U-SFQ, we must instantiate more U-SFQ than B-SFQ PEs to take advantage of parallelism, but in less aggregate area. How many more PEs and the resulting area reduction depend on the bit resolution. Figure 8(a) in blue shows the number of PEs necessary to get the same peak performance as a B-SFQ approach (812 TMAC/s) versus the number of bits. The number of PEs increases with the bit resolution, i.e., 4 bits of resolution requires 131K PEs, while 8 bits requires 2M PEs. Figure 8(a) also shows in orange the area gain calculated as $\frac{JJ^{\#_{U-SFQ}}}{JJ^{\#_{B-SFQ}}}$. With 4(8) bits of resolution, U-SFQ is 32× (2×) more area efficient than B-SFQ with the same peak performance.

3.2.2 Optimizing by Accuracy. The bit resolution affects the overall computation in two ways: (1) arithmetic accuracy and (2) classification accuracy.

(1) Arithmetic accuracy: Computing with U-SFQ yields the same computation accuracy than computing using a fixed point binary data representation [14]. This accuracy is limited only by quantization's effect, which is exacerbated when using a reduced number of bits. Remember that varying the bit resolution results in different computing latency, i.e., with 2 bits of resolution, we can represent four different numbers (Figure 2) and the computation epoch takes four time slots. With 4 bits, we can represent 16 different numbers (Figure 3) and the computing epoch takes 16 time slots. The fact that the computation is exact sets U-SFQ apart from approximate computing paradigms such as stochastic computing [6, 13].

On the other hand, U-SFQ shares with stochastic computing its robustness to noise and circuit non-ideal behavior. This is due to the uniform weight associated with pulses in pulse streams or time slots in RL. Previous work demonstrated that the effect of a missed/added pulse or RL jitter

is proportional to $1/2^b$ with b the number of bits. Moreover, the impact of this error becomes negligible in the computation result as b increases [14].

(2) Classification accuracy: Since U-SFQ yields the same computation accuracy as a fixed point binary data representation, U-SFQ yields the same classification accuracy as any CNN hardware accelerator that operates with quantized ifmap and weights. CNN inference is usually performed using floating-point (FP32) binary data. However, a wide breadth of work has been devoted to quantizing the ifmap and activation functions [37], such that there is no classification accuracy degradation when using 8 bit-fixed-point-binary data instead of FP32. Moreover, quantization enables aggressive reduction in the bit resolution without affecting the classification accuracy [9, 18, 19, 25, 39]. This bit reduction is crucial for U-SFQ success because operating with a reduced number of bits maximizes U-SFQ efficiency.

Eight bits of resolution is a popular choice because it yields top-1 accuracy degradation of only 0.1–0.4% for ResNet50, ResNet101, GoogleNet, Inception, Mobilenet, and Sufflenet CNN topologies [37]. However, for 8 bits, we need 2M U-SFQ PEs which yields 2× less area than B-SFQ. In that case, the utilization factor of such an array would be low due to the bandwidth roof imposed by off-chip memory access, as we will discuss below.

Fortunately, other approaches have demonstrated that it is possible to quantize weights and activation functions below 8 bits. Using a **Parameterized Clipping Activation (PACT)** method [8], there is an accuracy degradation of only 0.4–1 for ResNet18, Resnet20, Resnet50, and Alexnet and 4 bits of resolution. PACT uses a parameter α as a clipping level in the activation function, which is dynamically adjusted via gradient descent-based training. A caveat with PACT is that the first and last layers are calculated using FP32.

Other approaches convert pre-trained FP32 hyperparameters to quantized versions with reduced resolution. For instance, Banner et al. [3] achieves top-1 accuracy degradation, of 1.1–2.7% for VGG, VGG-BN, ResNet18, ResNet50, and ResNet101 using 4 bits in average. That is, some channels have higher/lower than 4 bit widths, but the total number of bits written to or read from memory is kept to an average of 4 bits. Note that with U-SFQ, different bit resolutions can be obtained by varying the epoch time. Furthermore, using an Incremental Network Quantization [54] method, there is only 0.15–1.6 Top1 classification accuracy degradation for Alexnet, GoogleNet, ResNet18, and ResNet50 using 5 bits.

We evaluate our U-SFQ accelerator performance by varying the bit resolution from 4 to 8. Since there have been promising results for 4 bits with small datasets, this is an active area of research in the CNN community. Five bits and a combination of 4 bits for the inner layers and 8 bits for the first and last layers (4,8) are cases of particular interest due to the success demonstrated by several bit reduction approaches [3, 8].

3.2.3 Optimizing by Memory Bandwidth. Limitations in the memory bandwidth prevent CNN hardware accelerators from achieving peak performance [20, 56]. This translates into idle PEs that wait for data from the off-chip memory or inefficient data mapping to the array causing low utilization factor. To understand the memory requirements of our proposed accelerator, we obtain the roofline plot shown in Figure 8(b) for different CNN benchmarks. We assume a bandwidth roof of 300 GBytes/s as supported by state-of-the-art High Bandwidth Memory technology [20, 21, 56]. Since the performance of a CNN hardware accelerator is data dependent, we calculate the performance in giga-multiply-accumulations per second GMAC/s (y-axis) and computation intensity in Bytes/MAC (x-axis) for five different benchmarks varying the array dimensions TR × TC × TM using exhaustive search.

Increasing TR and TC beyond 64 does not improve the overall computation performance. TR and TC are set by the ofmap row (R) and column (C) dimensions. R × C is large for the first CNN layers

(+200) but rapidly shrinks for the inner layers in a typical CNN topology. Figure 8(b) highlights in yellow the area of influence for TR × TC.

Increasing TM significantly improves the computation performance, especially that of the last CNN layers, which usually feature a large number of filters (+512). Figure 8(b) highlights in green the design space where increasing TM has effect. The maximum TM is bounded by memory bandwidth. Setting TM beyond 32 pushes the performance to the bandwidth roof, and beyond this limit, we obtain a large number of idle PEs per epoch.

3.3 Our Proposed U-SFQ CNN Hardware Accelerator Configuration

Based on the previous results, targeting an array with ~131 K PEs matches B-SFQ peak performance for 4 bits of resolution and maximizes area efficiency. Bandwidth limitations set the maximum TR = TC to 64. If using TR = TC = 64 TM maximum is set to 32. The remaining question is how do we organize these PEs?

We propose to use 16 small subarrays of $16 \times 16 \times 32$ PEs, as shown in Figure 7(a) and (b). In a naive implementation of a larger array, i.e., $64 \times 64 \times 32$, each weight is connected to 4,096 PEs, which involves a distribution network with at least 4,096 splitters per weight, adding $\approx$ 5 ps of delay per cell. In the small subarray each weight is connected to only 256 PEs. The 16 subarrays work in parallel to behave as a larger array as we discuss below.

3.3.1 Memory Banks. Each subarray has a dedicated ifmap, ofmap, and weight buffers. The distributed ifmap and ofmap buffers do not impact memory size or bandwidth because each buffer is just a bank of the overall on-chip memory structure. In contrast, the aggregated weight buffer is 16× larger than in a 2D-based accelerator. Each weight buffer keeps copies of the same weights to obtain larger TR × TC dimensions. Fortunately, this buffer is usually in the order of hundreds of kB. Using superNPU [20] as a baseline, the weight buffer for our implementation is 2 MB instead of 128 kB, which is within the on-chip memory budget.

3.3.2 Flexible Array. The initial layers of a CNN have large ofmap ($+100 \times 100$) and few channels (3). The final layers have small ofmap (-16×16) and many channels (+200). Given, the diverse set of layers dimensions, having smaller subarrays facilitates different array configurations, also known as modes, improving the PE utilization factor.

Figure 9 shows the total computing epochs (latency), stacking the epochs per layer, for different benchmarks. The best latency is obtained from the $16 \times 16 \times 512$ array because of the large numbers of channels (TM) that can be mapped to the array. This configuration benefits the last CNN layers the most (VGG16 highlights this the best). An $8 \times 8 \times 512$ has only 32K PEs, but it yields better latency for Resnet18, Resnet50, and Alexnet than a larger $64 \times 64 \times 32$ array with 131K PEs, showing how TM has a larger effect in latency than TR and TC. The $64 \times 64 \times 32$ array yields more aggregated latency but performs best for the larger ofmap of the first CNN layers.

Based on these results, we define two modes of operation (*A* and *B*). In *mode A*, the array operates as $64 \times 64 \times 32$. In *mode B*, the array operates as a $16 \times 16 \times 512$.

In *mode A* weights are broadcasted to all the weight buffers as shown in Figure 7(a). All the subarrays operate with the same weights and different inputs. Mode *A* benefits the initial layers with large ofmap sizes and fewer channels.

In *mode B* ifmap data are broadcasted to all the ifmap buffers in the subarrays as shown in Figure 7(b). All the subarrays operate with the same inputs yet different weights. Mode *B* benefits the computation throughput for the last layers with smaller ifmap sizes and more filters.

Mode *A* or *B* is only relevant while pre-loading the weight and ifmap buffers. The latency of the computation and loading remains the same because the duplicated data are broadcasted (Figure

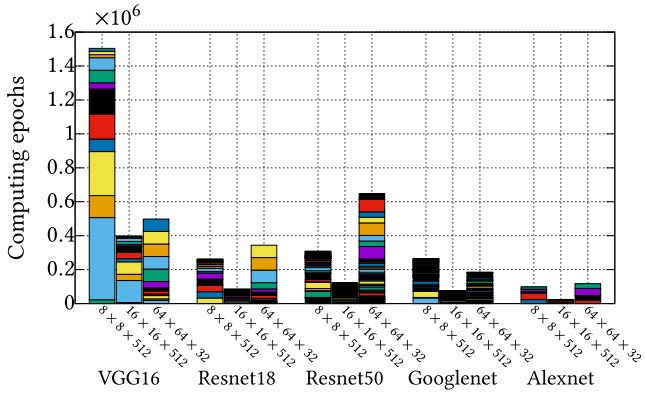


Fig. 9. Computing epochs per layer for three different array configurations. Each color represents a layer.

7(a) and (b)). We foresee the broadcasting hardware cost to be minimum (~15 splitters per signal) and do not significantly add to the overall cost of the system.

4 EVALUATION

We evaluate our proposed 3D U-SFQ PE array and calculate performance, area efficiency, and power efficiency and compare it with B-SFQ and CMOS approaches.

4.1 Methodology

We implement the proposed U-SFQ multiplier, accumulator, and PE using the open-source *MIT-LL SFQ5ee* 10 kA/cm^2 process. We run spice-level simulations with the open-source software WRSPICE to measure (1) dynamic and static power and (2) computation latency. We use the number of JJs as a metric for area.

For latency and throughput evaluation of our proposed CNN hardware accelerator, we build a behavioral model of the 3D array dataflow and memory accesses and evaluate the throughput for typical CNN benchmarks such as VGG16, ResNet18, ResNet50, GoogleNet, and Alexnet. These benchmarks can operate with a reduced bit width (4,5) without significant accuracy degradation.

We compare our U-SFQ accelerator with B-SFQ and CMOS. For the B-SFQ approach, we use a 256 × 64 array inspired by superNPU [20]. We build high-level models of state-of-the-art SFQ bit-pipelined multipliers and adders to obtain area and power metrics. For CMOS, we use a 256 × 256 PE array inspired in the TPU [21]. We run synthesis for CMOS multipliers, adders, and a PE to obtain power metrics, using the open-source OpenRoad synthesis tool [36]. To get latency and throughput metrics for the B-SFQ and CMOS PE arrays, we use Scale-Sim [41], an open-source CNN accelerator simulator that provides cycle-accurate timing.

For ERSFQ power evaluation we use the multiplication factors in Table 1 [33]. To include the cooling power we use a power efficiency of 400 W/W for a helium reliquefier refrigeration system [17, 20].

4.2 Performance

The performance of our proposed accelerator depends on the bit resolution. To evaluate latency and throughput, we run dataflow simulations for typical CNN benchmarks (Alexnet, Resnet, Googlenet, and VGG16), varying the number of bits from 4 to 8. We choose 4 bits as the low limit because for most benchmarks and quantization methods, using less than 4 bits results in unacceptable accuracy degradation. Figure 10(a) shows the speedup of our accelerator compared with the CMOS approach.

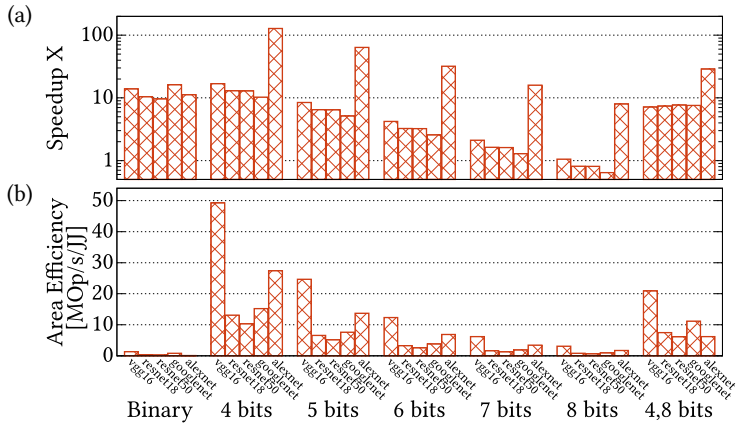


Fig. 10. (a) Speedup of SFQ over CMOS for binary and U-SFQ approaches. (b) Area efficiency calculated as throughput per JJ. Binary uses 8 bits. We vary the U-SFQ bit width from 4 to 8. (4,8) means variable bit width with first and last layers calculated with 8 bits and internal layers with 4 bits in average.

As expected, reducing the number of bits increases the speedup. Except for Alexnet, bit resolutions of 7 bits of less result in U-SFQ being faster than CMOS.

Using 5 bits of resolution in benchmarks such as Alexnet, Resnet18, and Googlenet yields a top-1[2] accuracy degradation of less than 1 [54] and speedups over CMOS of 63×, 6×, and 5×, respectively. Using a variable bit-width approach (4,8), Resnet50 and VGG16 [3] do not experience major accuracy degradation and yield 7× speedups over CMOS.

Figure 10(a) also shows the speedups of using a B-SFQ approach over CMOS for comparison. Note that U-SFQ yields comparable latency to B-SFQ for all the evaluated benchmarks if using 4 bits of resolution. Beyond 4 bits, B-SFQ results in better latency at the expense of 32× more area. Overall, reducing the computation latency in U-SFQ is achieved by increasing the number of PEs, that is, the parallelism, and keeping the bit resolution small.

4.3 Area

Table 3 compares the area (JJ count) and peak area efficiency (MAC/s/JJ) for our proposed U-SFQ PE array with a B-SFQ PE array. Our proposed PE array integrates 8× more PEs in 94% less area compared to the B-SFQ array. Figure 10(b) shows the area efficiency calculated as throughput per JJ, that is, mega-multiply-accumulations-per-second-per-JJ. For 8 bits, U-SFQ yields only 2–3× better area efficiency due to the long latency per computing epoch. As we reduce the bit resolution, the area efficiency improves. For 5 bits, the U-SFQ area efficiency is 18–173× better than binary. For a variable bit wdith (4,8) the area efficiency is 15–78× better than B-SFQ.

4.4 Power

Our proposed U-SFQ accelerator yields significant power savings compared with the B-SFQ approach because it has 96% fewer JJs. We extract the power associated with SFQ multipliers and adders and then extrapolate using the number of PEs in order to estimate the power consumed by the PE array. Table 4 reports the power for the proposed U-SFQ array and compares it with the B-SFQ and CMOS approaches.

[2]Model classification accuracy.

Table 3. Summary of Array Parameters for CNN Hardware Accelerators, including Frequency (Freq.), Peak Performance (Perf.), and Area Efficiency (Ef.)

	TPU	SuperNPU	U-SFQ
TC	256	256	64
TR	256	64	64
TM	1	1	32
PEs	65K	16K	131K
Number of JJs	-	297M	15.4M
Freq. (GHz)	0.7	50	50
Peak Perf. (TMAC/s)	45	819	409[a]
Area Ef. (MAC/s/JJ)	-	2.7M	29M[a]

[a]For 5 bits.

Table 4. Power Consumption (W) and Efficiency (TMAC/s/W) for Our Proposed U-SFQ CNN Array, CMOS Array, and B-SFQ Array

Tech.	Power (W)	TMAC/s/W
TPU array (CMOS)		
CMOS	24.32	1.89
B-SFQ array		
RSFQ w.o.c	172	4.7
RSFQ w.c.	67K	0.012
ERSFQ w.o.c	0.62	769
ERSFQ w.c.	210	1.9
U-SFQ array (5 bits)		
RSFQ w.o.c	3.9	105
RSFQ w.c.	1.6K	0.26
ERSFQ w.o.c	0.04	10 K
ERSFQ w.c.	16.2	25

Without cooling (w.o.c), and with cooling (w.c) assuming a 4K temperature.

Understanding the power consumption without including the cost of cooling is interesting because upcoming applications such as in open space or quantum computing already operate in supercooled environments and thus can readily benefit from superconducting computing. Without including the cost of cooling U-SFQ implemented with RSFQ gates reduces the power consumption by 44× and 6× compared with B-SFQ and CMOS, respectively. Using energy efficient (ERSFQ) gates U-SFQ results in 600× less power consumption than CMOS.

We also obtain the power efficiency in **tera-MAC-per-second-per-watt (TMAC/s/W)**, that is, how many multiply accumulations can the array perform per consumed watt. Figure 11 shows the power efficiency with and without cooling versus the bit resolution. Without including the cost of cooling, U-SFQ yields 44×–2× and 111×–6× more power efficiency than B-SFQ and CMOS,

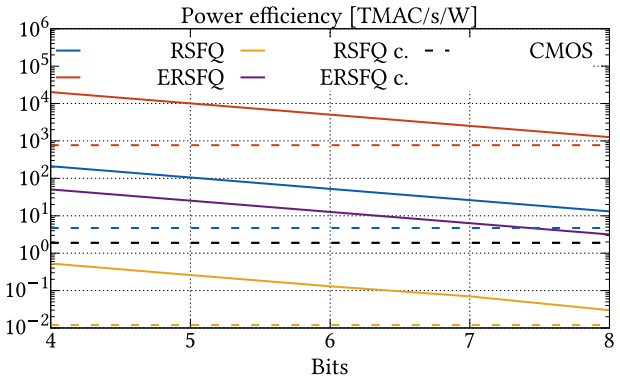


Fig. 11. Power efficiency for U-SFQ, B-SFQ, and CMOS. U-SFQ: continuous lines. B-SFQ: dashed lines. For SFQ, we include the cost of cooling.

respectively for 4–8 bits using RSFQ gates. Using ERSFQ, U-SFQ yields 26×–1.6× and 10 K×–667× better power efficiency than B-SFQ and CMOS, respectively.

Including the cost of cooling, RSFQ rapidly falls behind, yielding up to two orders of magnitude lower power efficiency than CMOS. Using ERSFQ gates, U-SFQ yields 26×–1.6× higher power efficiency than B-SFQ and CMOS.

5 CONCLUSION

U-SFQ is an alternative computing paradigm that leverages the high speed and low energy consumption of superconducting technologies while addressing the area stringent constraints that set back superconducting architectures. In this work, we extend U-SFQ and propose a novel CNN hardware accelerator. The proposed accelerator requires 96% less area than a state-of-the-art binary approach and yields 5–63× lower latency than CMOS without incurring major classification accuracy degradation. Including the cost of cooling, the proposed accelerator is 26× more power efficient than CMOS. These promising results make U-SFQ a prime candidate to unleash superconducting digital computing's potential.

ACKNOWLEDGMENTS

The authors would like to thank Darren Lyles and Meriam Bautista for helpful conversations and the anonymous reviewers for their valuable feedback.

REFERENCES

[1] Akira Akahori, Masamitsu Tanaka, Akito Sekiya, Akira Fujimaki, and Hisao Hayakawa. 2003. Design and demonstration of SFQ pipelined multiplier. *IEEE Transactions on Applied Superconductivity* 13, 2 (2003), 559–562. DOI: https://doi.org/10.1109/TASC.2003.813946

[2] Christopher L. Ayala, Tomoyuki Tanaka, Ro Saito, Mai Nozoe, Naoki Takeuchi, and Nobuyuki Yoshikawa. 2021. MANA: A Monolithic Adiabatic iNtegration Architecture microprocessor using 1.4-zJ/op unshunted superconductor Josephson junction devices. *IEEE Journal of Solid-State Circuits* 56, 4 (2021), 1152–1165. DOI: https://doi.org/10.1109/JSSC.2020.3041338

[3] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 32.

[4] Meriam Gay Bautista, Patricia Gonzalez-Guerrero, Darren Lyles, and George Michelogiannakis. 2021. Superconducting shuttle-flux shift buffer for race logic. In *Proceedings of the 2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, 796–801. DOI: https://doi.org/10.1109/MWSCAS47672.2021.9531899

[5] Paul Bunyk, Konstantin Likharev, and Dmitry Zinoviev. 2001. RSFQ technology: Physics and devices. *International Journal of High Speed Electronics and Systems* 11, 01 (2001), 257–305.

[6] Ruizhe Cai, Ao Ren, Olivia Chen, Ning Liu, Caiwen Ding, Xuehai Qian, Jie Han, Wenhui Luo, Nobuyuki Yoshikawa, and Yanzhi Wang. 2019. A stochastic-computing based deep learning framework using adiabatic quantum-flux-parametron superconducting technology. In *Proceedings of the 2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 567–578.

[7] Wei Chen, Alexander Rylyakov, Vaishal Patel, James E. Lukens, and Konstantin Likharev. 1999. Rapid single flux quantum T-flip flop operating up to 770 GHz. *IEEE Transactions on Applied Superconductivity* 9, 2 (1999), 3212–3215. DOI: https://doi.org/10.1109/77.783712

[8] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. PACT: Parameterized clipping activation for quantized neural networks. Retrieved from https://doi.org/10.48550/arXiv.1805.06085

[9] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. 2015. BinaryConnect: Training deep neural networks with binary weights during propagations. *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 28.

[10] Zidong Du, Robert Fasthuber, Tianshi Chen, Paolo Ienne, Ling Li, Tao Luo, Xiaobing Feng, Yunji Chen, and Olivier Temam. 2015. ShiDianNao: Shifting vision processing closer to the sensor. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 92–104.

[11] Timur V. Filippov, Anubhav Sahu, Alex F. Kirichenko, Igor V. Vernik, Mikhail Dorojevets, Christopher L. Ayala, and Oleg A. Mukhanov. 2012. 20 GHz operation of an asynchronous wave-pipelined RSFQ arithmetic-logic unit. *Physics Procedia* 36 (2012), 59–65. DOI: https://doi.org/10.1016/j.phpro.2012.06.130

[12] Akira Fujimaki, Masamitsu Tanaka, Ryo Kasagi, Katsumi Takagi, Masakazu Okada, Yuhi Hayakawa, Kensuke Takata, Hiroyuki Akaike, Nobuyuki Yoshikawa, Shuichi Nagasawa, Kazuyoshi Takagi, and Naofumi Takagi. 2014. Large-scale integrated circuit design based on a Nb nine-layer structure for reconfigurable data-path processors. *IEICE Transactions on Electronics* 97, 3 (2014), 157–165.

[13] Brian R. Gaines. 1967. Stochastic computing. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, 149–156.

[14] Patricia Gonzalez-Guerrero, Meriam Gay Bautista, Darren Lyles, and George Michelogiannakis. 2022. Temporal and SFQ pulse-streams encoding for area-efficient superconducting accelerators. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, 963–976. DOI: https://doi.org/10.1145/3503222.3507765

[15] Patricia Gonzalez-Guerrero and Mircea R. Stan. 2019. Asynchronous stochastic computing. In *Proceedings of the 2019 53rd Asilomar Conference on Signals, Systems, and Computers*. IEEE, 280–285.

[16] Patricia Gonzalez-Guerrero, Stephen G. Wilson, and Mircea R. Stan. 2019. Error-latency trade-off for asynchronous stochastic computing with ΣΔ streams for the IoT. In *Proceedings of the 2019 32nd IEEE International System-on-Chip Conference (SOCC)*. IEEE, 97–102.

[17] D. Scott Holmes, Andrew L. Ripple, and Marc A. Manheimer. 2013. Energy-efficient superconducting computing—Power budgets and requirements. *IEEE Transactions on Applied Superconductivity* 23, 3 (2013), 1701610–1701610.

[18] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 29.

[19] Kyuyeon Hwang and Wonyong Sung. 2014. Fixed-point feedforward deep neural network design using weights +1, 0, and −1. In *Proceedings of the 2014 IEEE Workshop on Signal Processing Systems (SiPS)*, 1–6. DOI: https://doi.org/10.1109/SiPS.2014.6986082

[20] Koki Ishida, Ilkwon Byun, Ikki Nagaoka, Kosuke Fukumitsu, Masamitsu Tanaka, Satoshi Kawakami, Teruo Tanimoto, Takatsugu Ono, Jangwoo Kim, and Koji Inoue. 2020. SuperNPU: An extremely fast neural processing unit using superconducting logic devices. In *Proceedings of the 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 58–72.

[21] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 1–12.

[22] Ryota Kashima, Ikki Nagaoka, Masamitsu Tanaka, Taro Yamashita, and Akira Fujimaki. 2021. 64-GHz datapath demonstration for bit-parallel SFQ microprocessors based on a gate-level-pipeline structure. *IEEE Transactions on Applied Superconductivity* 31, 5 (2021), 1–6.

[23] Fei Ke, Olivia Chen, Yanzhi Wang, and Nobuyuki Yoshikawa. 2021. Demonstration of a 47.8 GHz high-speed FFT processor using single-flux-quantum technology. *IEEE Transactions on Applied Superconductivity* 31, 5 (2021), 1–5.

[24] Asifullah Khan, Anabia Sohail, Umme Zahoora, and Aqsa Saeed Qureshi. 2020. A survey of the recent architectures of deep convolutional neural networks. *Artificial Intelligence Review* 53, 8 (2020), 5455–5516.

[25] Minje Kim and Paris Smaragdis. 2016. Bitwise neural networks. Retrieved from https://doi.org/10.48550/arXiv.1601.06071

[26] Gleb Krylov and Eby G. Friedman. 2022. *Single Flux Quantum Integrated Circuit Design*. Springer.

[27] Konstantin K. Likharev and Vasilii K. Semenov. 1991. RSFQ logic/memory family: A new Josephson-junction technology for sub-terahertz-clock-frequency digital systems. *IEEE Transactions on Applied Superconductivity* 1, 1 (1991), 3–28. DOI: https://doi.org/10.1109/77.80745

[28] Advait Madhavan, Timothy Sherwood, and Dmitri Strukov. 2014. Race logic: A hardware acceleration for dynamic programming algorithms. In *Proceeding of the 41st Annual International Symposium on Computer Architecuture (ISCA '14)*. IEEE Press, 517–528.

[29] Robert McDermott, Mikhail Grigorievich Vavilov, Benjamin Louis Thomas Plourde, Frank Karl Wilhelm, Peter James Liebermann, Oleg Alexandrovich Mukhanov, and Tatsuya Akira Ohki. 2018. Quantum–classical interface based on single flux quantum digital logic. *Quantum Science and Technology* 3, 2 (2018), 024004.

[30] Vratislav Michal, Emanuele Baggetta, Mario Aurino, Sophie Bouat, and Jean-Claude Villegier. 2011. Superconducting RSFQ logic: Towards 100GHz digital electronics. In *Proceedings of 21st International Conference Radioelektronika*, 1–8.

[31] Janardan Misra and Indranil Saha. 2010. Artificial neural networks in hardware: A survey of two decades of progress. *Neurocomputing* 74, 1 (2010), 239–255. DOI: https://doi.org/10.1016/j.neucom.2010.03.021

[32] Oleg A. Mukhanov, Vasili K. Semenov, and Konstantin K. Likharev. 1987. Ultimate performance of the RSFQ logic circuits. *IEEE Transactions on Magnetics* 23, 2 (1987), 759–762. DOI: https://doi.org/10.1109/TMAG.1987.1064951

[33] Oleg A. Mukhanov. 2011. Energy-efficient single flux quantum technology. *IEEE Transactions on Applied Superconductivity* 21, 3 (2011), 760–769.

[34] Oleg A. Mukhanov, Stanislav V. Polonsky, and Vasili K. Semenov. 1991. New elements of the RSFQ logic family. *IEEE Transactions on Magnetics* 27, 2 (1991), 2435–2438. DOI: https://doi.org/10.1109/20.133710

[35] Ikki Nagaoka, Masamitsu Tanaka, Koji Inoue, and Akira Fujimaki. 2019. A 48ghz 5.6 mw gate-level-pipelined multiplier using single-flux quantum logic. In *Proceedings of the 2019 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 460–462.

[36] OpenRoad. 2021. OpenRoad. Retrieved from https://theopenroadproject.org

[37] PyTorch. 2021. PyTorch. Retrieved from https://pytorch.org/docs/stable/quantization.html

[38] Atul Rahman, Jongeun Lee, and Kiyoung Choi. 2016. Efficient FPGA acceleration of convolutional neural networks using logical-3D compute array. In *Proceedings of the 2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1393–1398.

[39] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: Imagenet classification using binary convolutional neural networks. In *Proceedings of the European Conference on Computer Vision*. Springer, 525–542.

[40] Peter Russer. 1971. General energy relations for Josephson junctions. *Proceedings of the IEEE* 59, 2 (1971), 282–283.

[41] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. SCALE-Sim: Systolic CNN accelerator simulator. Retrieved from https://doi.org/10.48550/arXiv.1811.02883

[42] Catherine D. Schuman, Thomas E. Potok, Robert M. Patton, J. Douglas Birdwell, Mark E. Dean, Garrett S. Rose, and James S. Plank. 2017. A survey of neuromorphic computing and neural networks in hardware. Retrieved from https://doi.org/10.48550/arXiv.1705.06963

[43] Jing Shen, Haoqi Ren, Zhifeng Zhang, Jun Wu, Wenqi Pan, and Zhenyu Jiang. 2019. A high-performance systolic array accelerator dedicated for CNN. In *Proceedings of the 2019 IEEE 19th International Conference on Communication Technology (ICCT)*, 1200–1204. DOI: https://doi.org/10.1109/ICCT46805.2019.8947127

[44] Hyeonuk Sim and Jongeun Lee. 2017. A new stochastic computing multiplier with application to deep convolutional neural networks. In *Proceedings of the 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 1–6.

[45] Ivan Ivanovich Soloviev, Sergey Vladimirovich Bakurskiy, Vladimir Ivanovich Ruzhickiy, Nikolay Vladimirovich Klenov, Mikhail Yuryevich Kupriyanov, Andrey Anatolyevich Golubov, Olga Vladimirovna Skryabina, and Viktor Sergeyevich Stolyarov. 2021. Miniaturization of Josephson junctions for digital superconducting circuits. *Physical Review Applied* 16, 4 (2021), 044060.

[46] Vivienne Sze, Yu-Hsin Chen, Joel Emer, Amr Suleiman, and Zhengdong Zhang. 2017a. Hardware for machine learning: Challenges and opportunities. In *Proceedings of the 2017 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 1–8.

[47] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. 2017b. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE* 105, 12 (2017), 2295–2329.

[48] Swamit S. Tannu, Douglas M. Carmean, and Moinuddin K. Qureshi. 2017. Cryogenic-DRAM based memory system for scalable quantum computers: A feasibility study. In *Proceedings of the International Symposium on Memory Systems (MEMSYS '17)*. Association for Computing Machinery, New York, NY, 189–195. DOI: https://doi.org/10.1145/3132402.3132436

[49] Swamit S. Tannu, Poulami Das, Michael L. Lewis, Robert Krick, Douglas M. Carmean, and Moinuddin K. Qureshi. 2019. A case for superconducting accelerators. In *Proceedings of the 16th ACM International Conference on Computing Frontiers*, 67–75. DOI: https://doi.org/10.1145/3310273.3321561

[50] Georgios Tzimpragos, Dilip Vasudevan, Nestan Tsiskaridze, George Michelogiannakis, Advait Madhavan, Jennifer Volk, John Shalf, and Timothy Sherwood. 2020. A computational temporal logic for superconducting accelerators. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*. Association for Computing Machinery, New York, NY, 435–448. DOI: https://doi.org/10.1145/3373376.3378517

[51] T. Van Duzer. 2005. Cryogenic memories for RSFQ ultra-high-speed processor. In *Proceedings of the 2005 ACM/IEEE Conference on Supercomputing (SC '05)*. IEEE Computer Society, 66. DOI: https://doi.org/10.1109/SC.2005.21

[52] Mark H. Volkmann, Anubhav Sahu, Coenrad J. Fourie, and Oleg A. Mukhanov. 2013. Experimental investigation of energy-efficient digital circuits based on eSFQ logic. *IEEE Transactions on Applied Superconductivity* 23, 3 (2013), 1301505–1301505. DOI: https://doi.org/10.1109/TASC.2013.2240755

[53] Di Wu, Jingjie Li, Ruokai Yin, Hsuan Hsiao, Younghyun Kim, and Joshua San Miguel. 2020. UGEMM: Unary computing architecture for GEMM applications. In *Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 377–390.

[54] Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. 2017. Incremental network quantization: Towards lossless cnns with low-precision weights. Retrieved from https://doi.org/10.48550/arXiv.1702.03044

[55] Dmitry Zinoviev. 2021. RFSQ Cell Library. Retrieved from http://www.physics.sunysb.edu/Physics/RSFQ/Lib/contents.html

[56] Farzaneh Zokaee and Lei Jiang. 2021. SMART: A heterogeneous scratchpad memory architecture for superconductor SFQ-based systolic CNN accelerators. *Proceedings of the MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (October 2021). DOI: https://doi.org/10.1145/3466752.3480041

Received 27 April 2023; revised 2 October 2023; accepted 8 March 2024